



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 9
по курсу «Языки и методы программирования»
«Перегрузка операций»

Студент группы ИУ9-21Б Белянин Г. И.

Преподаватель Посевин Д. П.

Москва 2026

1 Цель

Данная работа предназначена для изучения возможностей языка C++, обеспечивающих применение знаков операций к объектам пользовательских типов.

2 Персональный вариант

`LazyMatrix<T>` – матрица с элементами типа `T` неопределённого размера, растущая по мере надобности. Требование к типу `T`: наличие конструктора по умолчанию. Операции, которые должны быть перегружены для `LazyMatrix<T>`:

«`( )`» – получение ссылки элемент с индексами `i` и `j` (размер матрицы должен быть автоматически увеличен, если `i` или `j` выходят за её пределы); `m` и `n` – возвращают количество строк и столбцов, соответственно; «`!`» – транспонирование матрицы (возвращает новую матрицу). «`==`», «`!=`».

3 Практическая реализация

Main

```
#include <iostream>
#include <vector>

template <typename T>
class LazyMatrix {
private:
    std::vector<std::vector<T>> data;

    void grow_to(size_t rows, size_t cols) {
        if (rows > m()) {
            data.resize(rows, std::vector<T>(n()));
        }
        if (cols > n()) {
            for (auto& row : data) {
                row.resize(cols);
            }
        }
    }

public:
    LazyMatrix() = default;

    size_t m() const { return data.size(); }
    size_t n() const { return data.empty() ? 0 : data[0].size(); }

    T& operator()(size_t i, size_t j) {
        grow_to(i + 1, j + 1);
        return data[i][j];
    }

    T operator()(size_t i, size_t j) const {
        if (i >= m() || j >= n()) return T();
        return data[i][j];
    }

    LazyMatrix<T> operator!() const {
        LazyMatrix<T> res;
        size_t current_m = m();
        size_t current_n = n();
```

```

        if (current_m > 0 && current_n > 0) {
            res.grow_to(current_n, current_m);
            for (size_t i = 0; i < current_m; ++i) {
                for (size_t j = 0; j < current_n; ++j) {
                    res.data[j][i] = data[i][j];
                }
            }
        }
        return res;
    }

    bool operator==(const LazyMatrix<T>& other) const {
        if (m() != other.m() || n() != other.n()) return false;
        return data == other.data;
    }

    bool operator!=(const LazyMatrix<T>& other) const {
        return !(*this == other);
    }
};

int main() {
    LazyMatrix<int> matrix;
    matrix(1, 1) = 5;
    matrix(0, 2) = 10;

    std::cout << "Original: " << matrix.m() << "x" << matrix.n() << std::endl
                ;

    LazyMatrix<int> transposed = !matrix;

    std::cout << "Transposed: " << transposed.m() << "x" << transposed.n() <<
                std::endl;
    std::cout << "Value at (2,0): " << transposed(2, 0) << " (must be 10)" <<
                std::endl;

    return 0;
}

```

4 Результаты

Original: 2x3

Transposed: 3x2

Value at (2,0): 10 (must be 10)

Program ended with exit code: 0

5 Выводы

В ходе выполнения лабораторной работы был разработан шаблон класса `LazyMatrix<T>`, реализующий динамическую матрицу неопределенного размера, расширяющуюся по мере обращения к элементам. Освоены механизмы перегрузки операций в C++ для пользовательских типов данных.